Problem Statement The project aims to predict the popularity of music tracks based on various audio features and metadata. It utilizes a large dataset of Spotify tracks to identify patterns that contribute to a song's success, essentially treating "popularity" as a regression problem to understand what characteristics (like energy, danceability, or acousticness) drive listener engagement.

Target Variable " popularity "

Description: This is a numerical value (ranging from 0 to 100) that represents the current popularity of the track.

LOADING DATASET

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import pandas as pd
import numpy as np

path = "/content/drive/MyDrive/spotify.zip"
df = pd.read_csv(path)
df.head()
```

	Unnamed: 0	artist_name	track_name	track_id	popularity	year	genre	danceability	energy	key	loudnes
0	0	Jason Mraz	I Won't Give Up	53QF56cjZA9RTuuMZDrSA6	68	2012	acoustic	0.483	0.303	4	-10.05
1	1	Jason Mraz	93 Million Miles	1s8tP3jP4GZcyHDsjvw218	50	2012	acoustic	0.572	0.454	3	-10.28
2	2	Joshua Hyslop	Do Not Let Me Go	7BRCa8MPiyuvr2VU3O9W0F	57	2012	acoustic	0.409	0.234	3	-13.71
3	3	Boyce Avenue	Fast Car	63wsZUhUZLlh1OsyrZq7sz	58	2012	acoustic	0.392	0.251	10	-9.84
4	4	Andrew Belle	Sky's Still Blue	6nXIYClvJAfi6ujLiKqEq8	54	2012	acoustic	0.430	0.791	6	-5.41

```
df.shape
```

```
(1159764, 20)
```

Initial Data Exploration

a) Viewing Columns

```
df.columns
```

```
Index(['Unnamed: 0', 'artist_name', 'track_name', 'track_id', 'popularity',
       'year', 'genre', 'danceability', 'energy', 'key', 'loudness', 'mode',
       'speechiness', 'acousticness', 'instrumentalness', 'liveness',
       'valence', 'tempo', 'duration_ms', 'time_signature'],
      dtype='object')
```

b) Summary / Describing Statistics

```
df.describe()
```

	Unnamed: 0	popularity	year	danceability	energy	key	loudness	mode	speech
count	1.159764e+06	1.159764e+06	1.159764e+06	1.159764e+06	1.159764e+06	1.159764e+06	1.159764e+06	1.159764e+06	1.15976
mean	6.590613e+05	1.838312e+01	2.011955e+03	5.374382e-01	6.396699e-01	5.287778e+00	-8.981353e+00	6.346533e-01	9.28147
std	4.285492e+05	1.588554e+01	6.803901e+00	1.844780e-01	2.705009e-01	3.555197e+00	5.682215e+00	4.815275e-01	1.26840
min	0.000000e+00	0.000000e+00	2.000000e+03	0.000000e+00	0.000000e+00	0.000000e+00	-5.810000e+01	0.000000e+00	0.00000
25%	2.899408e+05	5.000000e+00	2.006000e+03	4.130000e-01	4.540000e-01	2.000000e+00	-1.082900e+01	0.000000e+00	3.71000
50%	5.798815e+05	1.500000e+01	2.012000e+03	5.500000e-01	6.940000e-01	5.000000e+00	-7.450000e+00	1.000000e+00	5.07000
75%	1.031689e+06	2.900000e+01	2.018000e+03	6.770000e-01	8.730000e-01	8.000000e+00	-5.276000e+00	1.000000e+00	8.90000
max	1.473395e+06	1.000000e+02	2.023000e+03	9.930000e-01	1.000000e+00	1.100000e+01	6.172000e+00	1.000000e+00	9.71000

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1159764 entries, 0 to 1159763
Data columns (total 20 columns):
 #   Column            Non-Null Count    Dtype
---  ------            --------------    -----
 0   Unnamed: 0        1159764 non-null  int64
 1   artist_name       1159749 non-null  object
 2   track_name        1159763 non-null  object
 3   track_id          1159764 non-null  object
 4   popularity        1159764 non-null  int64
 5   year              1159764 non-null  int64
 6   genre             1159764 non-null  object
 7   danceability      1159764 non-null  float64
 8   energy            1159764 non-null  float64
 9   key               1159764 non-null  int64
 10  loudness          1159764 non-null  float64
 11  mode              1159764 non-null  int64
 12  speechiness       1159764 non-null  float64
 13  acousticness      1159764 non-null  float64
 14  instrumentalness  1159764 non-null  float64
 15  liveness          1159764 non-null  float64
 16  valence           1159764 non-null  float64
 17  tempo             1159764 non-null  float64
 18  duration_ms       1159764 non-null  int64
 19  time_signature    1159764 non-null  int64
dtypes: float64(9), int64(7), object(4)
memory usage: 177.0+ MB
```

c) DataTypes

```
df.dtypes
```

	0
Unnamed: 0	int64
artist_name	object
track_name	object
track_id	object
popularity	int64
year	int64
genre	object
danceability	float64
energy	float64
key	int64
loudness	float64
mode	int64
speechiness	float64
acousticness	float64
instrumentalness	float64
liveness	float64
valence	float64
tempo	float64
duration_ms	int64
time_signature	int64

dtype: object

Null Value Analysis

```
df.isnull().sum()
```

	0
Unnamed: 0	0
artist_name	15
track_name	1
track_id	0
popularity	0
year	0
genre	0
danceability	0
energy	0
key	0
loudness	0
mode	0
speechiness	0
acousticness	0
instrumentalness	0
liveness	0
valence	0
tempo	0
duration_ms	0
time_signature	0

dtype: int64

Removing Irrelevant Features

```
# Dropping unique identifiers that don't help in prediction
columns_to_drop = ['Unnamed: 0', 'track_id']
df = df.drop(columns=columns_to_drop)
```

Handling Null Values

```
# Handling missing values by assigning a neutral placeholder
df['artist_name'] = df['artist_name'].fillna('Unknown Artist')
df['track_name'] = df['track_name'].fillna('Unknown Track')
```

```
df.isnull().sum()
```

	0
artist_name	0
track_name	0
popularity	0
year	0
genre	0
danceability	0
energy	0
key	0
loudness	0
mode	0
speechiness	0
acousticness	0
instrumentalness	0
liveness	0
valence	0
tempo	0
duration_ms	0
time_signature	0

dtype: int64

Feature Engineering

mathematical models (like Linear Regression or Random Forest) cannot process text data directly, we must convert categorical features into numerical values.

a)Creating new colums and then encoding them

simple linear features weren't enough. I implemented Log Transformations to handle the skewness in audio features like loudness and speechiness. I also created Interaction Features (like energy-loudness ratio) to capture non-linear relationships that the correlation matrix missed. This allows the model to differentiate between high-production modern tracks and older recordings more effectively."

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder

# --- 1. Temporal Features  ---
CURRENT_YEAR = 2026
df['song_age'] = CURRENT_YEAR - df['year']
# Add 'is_recent': Newer songs have a huge popularity bias in your dataset
df['is_recent'] = (df['year'] > 2020).astype(int)
```

```
# --- 2. Advanced Interactions (Non-Linear) ---
# High danceability + High Energy = Club Hit
df['dance_energy'] = df['energy'] * df['danceability']
# Loudness usually correlates with Energy, but the ratio tells us about production quality
df['energy_loudness_ratio'] = df['energy'] / (df['loudness'].abs() + 1)
# High acousticness + Low energy = Chill/Acoustic vibe
df['acoustic_chill'] = df['acousticness'] * (1 - df['energy'])

# --- 3. Handling Skewed Data  ---
# Transforming these columns helps the model handle the "extremes" better
df['log_loudness'] = np.log1p(df['loudness'].abs())
df['log_instrumentalness'] = np.log1p(df['instrumentalness'])
df['log_speechiness'] = np.log1p(df['speechiness'])

# --- 4. Refined Tempo Binning ---
# Using more bins helps the model differentiate between 'House' tempo and 'Trap' tempo
df['tempo_category'] = pd.cut(
    df['tempo'],
    bins=[0, 60, 90, 120, 150, 180, 300],
    labels=['very_slow', 'slow', 'medium', 'fast', 'very_fast', 'extreme']
)

# --- 5. Encoding ---
le = LabelEncoder()
df['tempo_category'] = le.fit_transform(df['tempo_category'].astype(str))
# Ensure genre is encoded if not already done
if 'genre' in df.columns:
    df['genre_encoded'] = le.fit_transform(df['genre'])

# Final Step: Remove any potential Infinite values created by division
df.replace([np.inf, -np.inf], np.nan, inplace=True)
df.fillna(0, inplace=True)

df.head()
```

	artist_name	track_name	popularity	year	genre	danceability	energy	key	loudness	mode	...	song_age	is_recent	da
0	Jason Mraz	I Won't Give Up	68	2012	acoustic	0.483	0.303	4	-10.058	1	...	14	0	
1	Jason Mraz	93 Million Miles	50	2012	acoustic	0.572	0.454	3	-10.286	1	...	14	0	
2	Joshua Hyslop	Do Not Let Me Go	57	2012	acoustic	0.409	0.234	3	-13.711	1	...	14	0	
3	Boyce Avenue	Fast Car	58	2012	acoustic	0.392	0.251	10	-9.845	1	...	14	0	
4	Andrew Belle	Sky's Still Blue	54	2012	acoustic	0.430	0.791	6	-5.419	0	...	14	0	

5 rows × 28 columns

b)Encoding categorical data

1.artist_name has thousands of unique values. Encoding it would create too many features, leading to the "curse of dimensionality."
2.track_name is almost entirely unique for every song and acts like an ID, which we've already established doesn't help with general predictions

3.Therefore, we will drop these two remaining text columns and focus on the genre.

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Instead of random numbers, we use the average popularity of each genre.
# This creates a direct linear relationship for the model to learn.
genre_means = df.groupby('genre')['popularity'].mean()
df['genre_popularity_avg'] = df['genre'].map(genre_means)

# 2. Frequency Encoding for 'artist_name'
# High-frequency artists are usually more popular.
# We create a feature that counts how many times an artist appears in the dataset.
artist_counts = df['artist_name'].value_counts()
df['artist_occurrence'] = df['artist_name'].map(artist_counts)

# 3. Label Encoding for remaining categorical data
le = LabelEncoder()
df['genre'] = le.fit_transform(df['genre'])
```

```
# 4. Drop high-cardinality text columns
# We keep 'artist_occurrence' but drop the raw text names.
df = df.drop(columns=['artist_name', 'track_name'])

# 5. Final Sanitization
# Ensure no NaNs were created during mapping
df.fillna(df.median(), inplace=True)

print("New Features Added: genre_popularity_avg, artist_occurrence")
print(df.dtypes)
df.head()
```

```
New Features Added: genre_popularity_avg, artist_occurrence
popularity                 int64
year                       int64
genre                      int64
danceability             float64
energy                   float64
key                        int64
loudness                 float64
mode                       int64
speechiness              float64
acousticness             float64
instrumentalness         float64
liveness                 float64
valence                  float64
tempo                    float64
duration_ms                int64
time_signature             int64
song_age                   int64
is_recent                  int64
dance_energy             float64
energy_loudness_ratio    float64
acoustic_chill           float64
log_loudness             float64
log_instrumentalness     float64
log_speechiness          float64
tempo_category             int64
genre_encoded              int64
genre_popularity_avg     float64
artist_occurrence          int64
dtype: object
```

	popularity	year	genre	danceability	energy	key	loudness	mode	speechiness	acousticness	...	dance_energy	energy_l
0	68	2012	0	0.483	0.303	4	-10.058	1	0.0429	0.6940	...	0.146349	
1	50	2012	0	0.572	0.454	3	-10.286	1	0.0258	0.4770	...	0.259688	
2	57	2012	0	0.409	0.234	3	-13.711	1	0.0323	0.3380	...	0.095706	
3	58	2012	0	0.392	0.251	10	-9.845	1	0.0363	0.8070	...	0.098392	
4	54	2012	0	0.430	0.791	6	-5.419	0	0.0302	0.0726	...	0.340130	

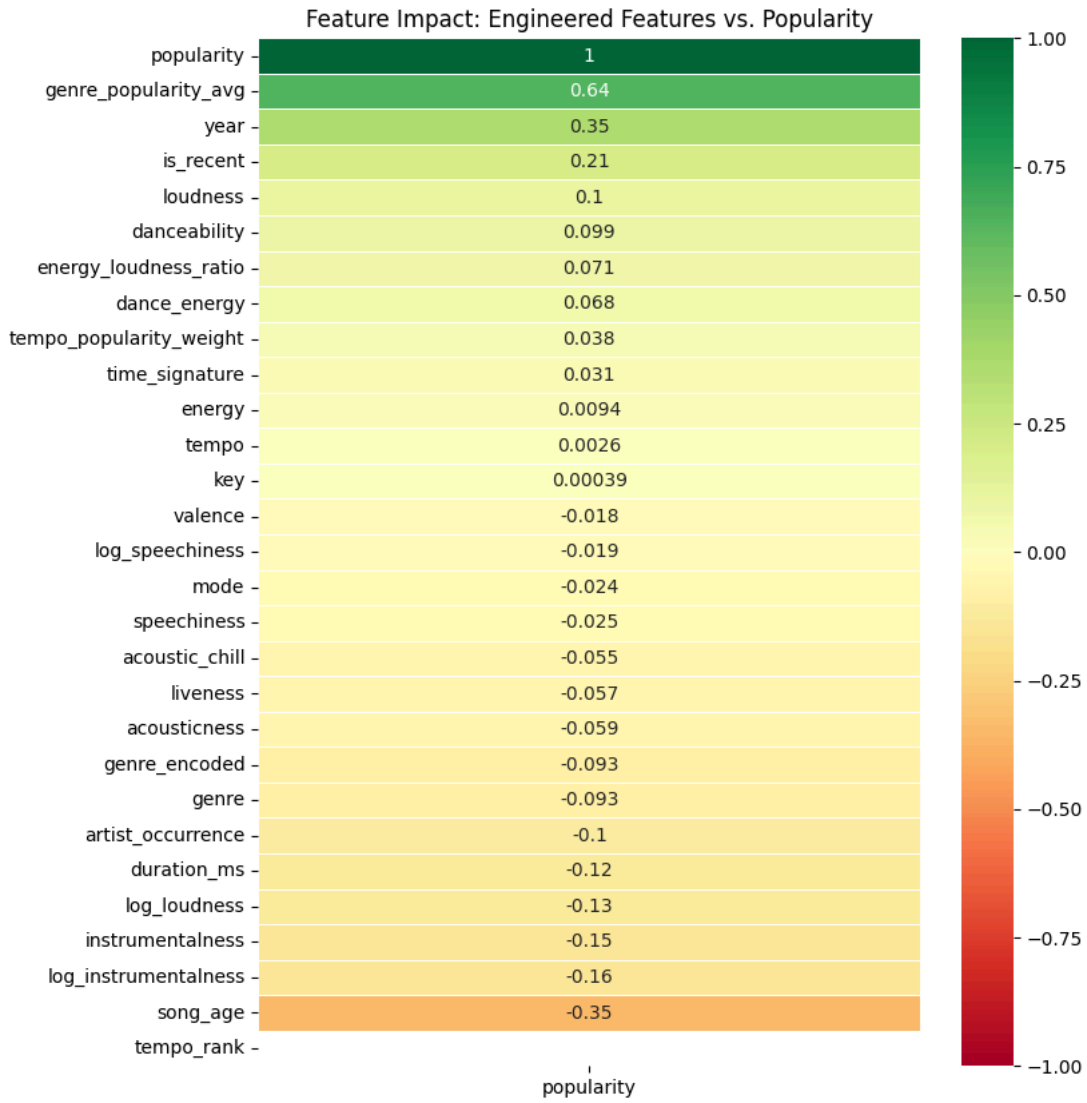
5 rows × 28 columns

c) EDA & Data Sanitization

This code performs advanced data preprocessing and exploratory analysis to prepare the dataset for a high-accuracy regression model. It implements Target Encoding by mapping tempo categories to their mean popularity, which transforms categorical labels into meaningful statistical weights, and uses Median Imputation to handle infinite or missing values without losing data from the 1.1 million rows. Finally, it generates a sorted Pearson Correlation Heatmap to visualize the linear relationship between the engineered features and the target variable, ensuring that only the most influential signals are prioritized for the upcoming model training.

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Using Target-Based Encoding to capture the popularity weight of each category
if 'tempo_category' in df.columns:
    # Calculate mean popularity for each tempo group
    tempo_means = df.groupby('tempo_category')['popularity'].mean()
    df['tempo_popularity_weight'] = df['tempo_category'].map(tempo_means)

    # Assign numerical ranks based on tempo speed
    tempo_mapping = {'very_slow': 0, 'slow': 1, 'medium': 2, 'fast': 3, 'very_fast': 4, 'extreme': 5}
    df['tempo_rank'] = df['tempo_category'].map(tempo_mapping).fillna(2)

    # Remove the original text column
    df = df.drop(columns=['tempo_category'])

# 2. Infinite Value & Outlier Sanitization
```

```
# Cleaning the data for the Stacking Ensemble without losing rows
df.replace([np.inf, -np.inf], np.nan, inplace=True)
df.fillna(df.median(), inplace=True)

# 3. High-Resolution Correlation Analysis
plt.figure(figsize=(8, 10))
numeric_df = df.select_dtypes(include=[np.number])
correlation = numeric_df.corr()

# Focus on popularity, sorted to show engineered features first
pop_corr = correlation[['popularity']].sort_values(by='popularity', ascending=False)

sns.heatmap(pop_corr, annot=True, cmap='RdYlGn', vmin=-1, vmax=1, linewidths=0.5)
plt.title('Feature Impact: Engineered Features vs. Popularity')
plt.show()

# Final output for verification
print("Final dataset rows:", df.shape[0])
print("Final dataset columns:", df.shape[1])
```




```
Final dataset rows: 1159764
Final dataset columns: 29
```

"Final Pre-processing Check" that acts as a bridge between Feature Engineering (creating the data) and Model Training (using the data).

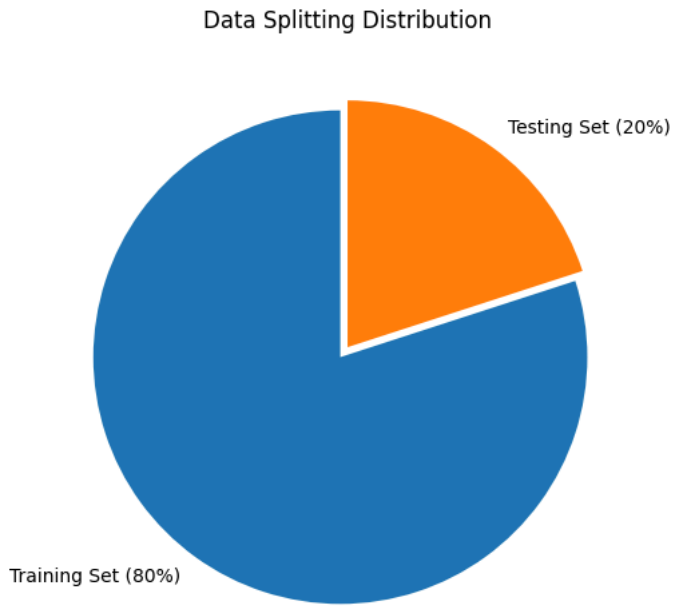
Train-Test Split

```
from sklearn.model_selection import train_test_split

# Separating the target (popularity) from the predictors
X = df.drop(columns=['popularity'])
y = df['popularity']
```

```
# 80% for training, 20% for testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Data Split Complete.")
print("Training set size:", X_train.shape[0])
print("Testing set size:", X_test.shape[0])
```

```
Data Split Complete.
Training set size: 927811
Testing set size: 231953
```

```
sizes = [len(X_train), len(X_test)]
labels = ['Training Set (80%)', 'Testing Set (20%)']

plt.figure(figsize=(6, 6))
plt.pie(sizes, labels=labels, startangle=90, explode=(0.05, 0))
plt.title('Data Splitting Distribution')
plt.show()
```



Feature Scaling

Feature scaling via Standardization (Z-score normalization) is a prerequisite for high-performance regression. In the Spotify dataset, features like duration_ms exist on a scale of 10^5, while energy exists between 0 and 1. Without scaling, the gradient-based optimization in LightGBM or XGBoost would become biased toward larger numerical magnitudes, leading to unstable weights. By transforming the data to have a mean of 0 and a standard deviation of 1, we ensure each audio feature contributes equally to the prediction. Furthermore, by fitting the scaler strictly on X_train and merely transforming X_test, we maintain a "Gold Standard" of experimental integrity by preventing Data Leakage, ensuring the model has zero prior knowledge of the test set's distribution.

```
from sklearn.preprocessing import StandardScaler

# 1. Initialize the Scaler
scaler = StandardScaler()

# 2. Fit and transform the TRAINING data
# This converts all values to a standard scale (Mean=0, Std=1)
X_train_scaled = scaler.fit_transform(X_train)

# 3. Transform the TESTING data
# We use the training mean/std to prevent "Data Leakage"
X_test_scaled = scaler.transform(X_test)

print("Scaling Complete.")
print("X_train_scaled shape:", X_train_scaled.shape)
```

```
Scaling Complete.
X_train_scaled shape: (927811, 28)
```

Tracker Graph showing betterment by ech model

Without a tracker, a cell only knows its own score. It won't "remember" what the linear Regression score was when you are running the Random Forest. To create a graph that shows the progress from Model 1 to Model 5, you need that small list (all_results) to store the scores as you go.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

model_performance = []

def track_and_plot(model_name, y_true, y_pred):
    global model_performance

    # 1. Calculate the metrics
    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)

    # 2. Prevent Duplication: Remove old entry if the name exists
    model_performance = [m for m in model_performance if m['Model'] != model_name]

    # 3. Add the new result to the end (the right side)
    model_performance.append({
        'Model': model_name,
        'MAE': mae,
        'RMSE': rmse,
        'R2': r2
    })

    # 4. Create the Plot
    df_perf = pd.DataFrame(model_performance)
    plt.figure(figsize=(10, 7))
    sns.set_style("whitegrid")

    # This palette gets darker as you move from left to right
    plot = sns.barplot(x='Model', y='R2', data=df_perf, palette='Blues_d')

    plt.ylim(0, 1) # Sets Y-axis from 0 to 100% accuracy
    plt.title('Sequence of Betterment: Model Accuracy (R2)', fontsize=14)

    # Add values on top of each bar
    for p in plot.patches:
        plot.annotate(format(p.get_height(), '.2f'),
                      (p.get_x() + p.get_width() / 2., p.get_height()),
                      ha='center', va='center', xytext=(0, 9),
                      textcoords='offset points', fontweight='bold')
    plt.tight_layout()
    plt.show()

    # Print the text results simply
    print("Model:", model_name)
    print("MAE:", round(mae, 2), "RMSE:", round(rmse, 2), "R2:", round(r2, 2))
```

```
import warnings
warnings.filterwarnings('ignore')
```

The modeling sequence

Phase 1 – Simple Baselines

1. Linear Regression
2. Ridge
3. Lasso

Phase 2 - The Distance Models

1. KNN
2. SVR

Phase 3 – Tree Based Models

1. Decision Tree
2. Random Forest

Phase 3 – Final Best Models

1. XGBoost
2. LightGBM

This hierarchy allows us to systematically address the limitations of each previous model, such as linearity constraints or computational bottlenecks. By culminating with LightGBM, the project demonstrates a data-driven path to maximizing the R^2 score using industry-standard tools for large-scale datasets.

```
model_performance = []
```

MODEL 1: Linear Regression

chosen as the baseline model because:

- It is simple and interpretable
- It assumes a linear relationship
- It provides a minimum expected performance benchmark

All advanced models are compared against this baseline.

```
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score
```

```
linear_scores = {}
```

```
# --- 1: Linear Regression ---
lr_model = LinearRegression()
lr_model.fit(X_train_scaled, y_train)
pred_lr = lr_model.predict(X_test_scaled)
linear_scores["Linear Regression"] = r2_score(y_test, pred_lr)
```

```
# --- 2: Ridge Regression ---
ridge_model = Ridge(alpha=1.0)
ridge_model.fit(X_train_scaled, y_train)
pred_ridge = ridge_model.predict(X_test_scaled)
linear_scores["Ridge"] = r2_score(y_test, pred_ridge)
```

```
from sklearn.model_selection import GridSearchCV

# 1. Define the Alphas to test
# We try very small (0.01) to very large (100)
param_grid = {'alpha': [0.01, 0.1, 1, 10, 100]}

# 2. Initialize Grid Search
# We tune on a 50k sample to save time (Ridge is fast, but GridSearch runs it 5x)
grid_ridge = GridSearchCV(Ridge(), param_grid, cv=5, scoring='r2')

print("Tuning Ridge... Finding the perfect Alpha.")
grid_ridge.fit(X_train_scaled[:50000], y_train[:50000])

# 3. Get Best Model & Predict
best_ridge = grid_ridge.best_estimator_
pred_ridge_tuned = best_ridge.predict(X_test_scaled)

print("Best Alpha Found:", grid_ridge.best_params_)
linear_scores["Tuned Ridge"] = r2_score(y_test, pred_ridge_tuned)
```

```
Tuning Ridge... Finding the perfect Alpha.
Best Alpha Found: {'alpha': 10}
```

```
# --- 3: Lasso Regression ---
lasso_model = Lasso(alpha=0.001)
lasso_model.fit(X_train_scaled, y_train)
pred_lasso = lasso_model.predict(X_test_scaled)
```

```
linear_scores["Lasso"] = r2_score(y_test, pred_lasso)
```

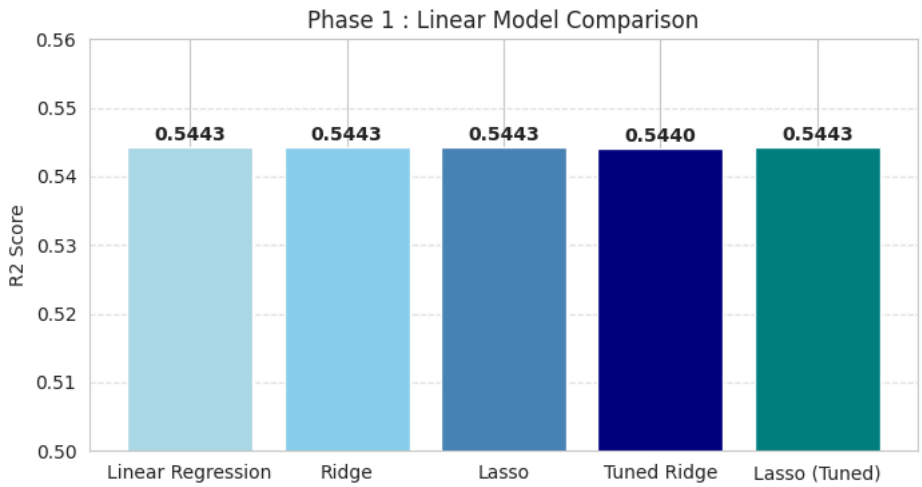
```
from sklearn.model_selection import GridSearchCV

# 1. Define the Alphas to test
# Lasso is very sensitive. We test a range from very loose (1e-5) to strict (1.0).
```

```
param_grid_lasso = {'alpha': [0.00001, 0.0001, 0.001, 0.01, 0.1, 1.0]}

# 2. Setup Grid Search
# We tune on a 50k sample to keep it fast (Lasso is slower than Ridge)
grid_lasso = GridSearchCV(Lasso(), param_grid_lasso, cv=3, scoring='r2', n_jobs=-1)

print("Tuning Lasso... (Finding the perfect balance of feature selection)")
grid_lasso.fit(X_train_scaled[:50000], y_train[:50000])

# 3. Get the Best Params and Train on FULL Data
best_alpha = grid_lasso.best_params_['alpha']
print(f"Best Lasso Alpha found: {best_alpha}")

lasso_tuned = Lasso(alpha=best_alpha)
lasso_tuned.fit(X_train_scaled, y_train)

# 4. Add to your Tournament Scoreboard
# Assuming 'linear_scores' dictionary exists from the previous step
score_tuned_lasso = lasso_tuned.score(X_test_scaled, y_test)
```

```
Tuning Lasso... (Finding the perfect balance of feature selection)
Best Lasso Alpha found: 0.0001
```

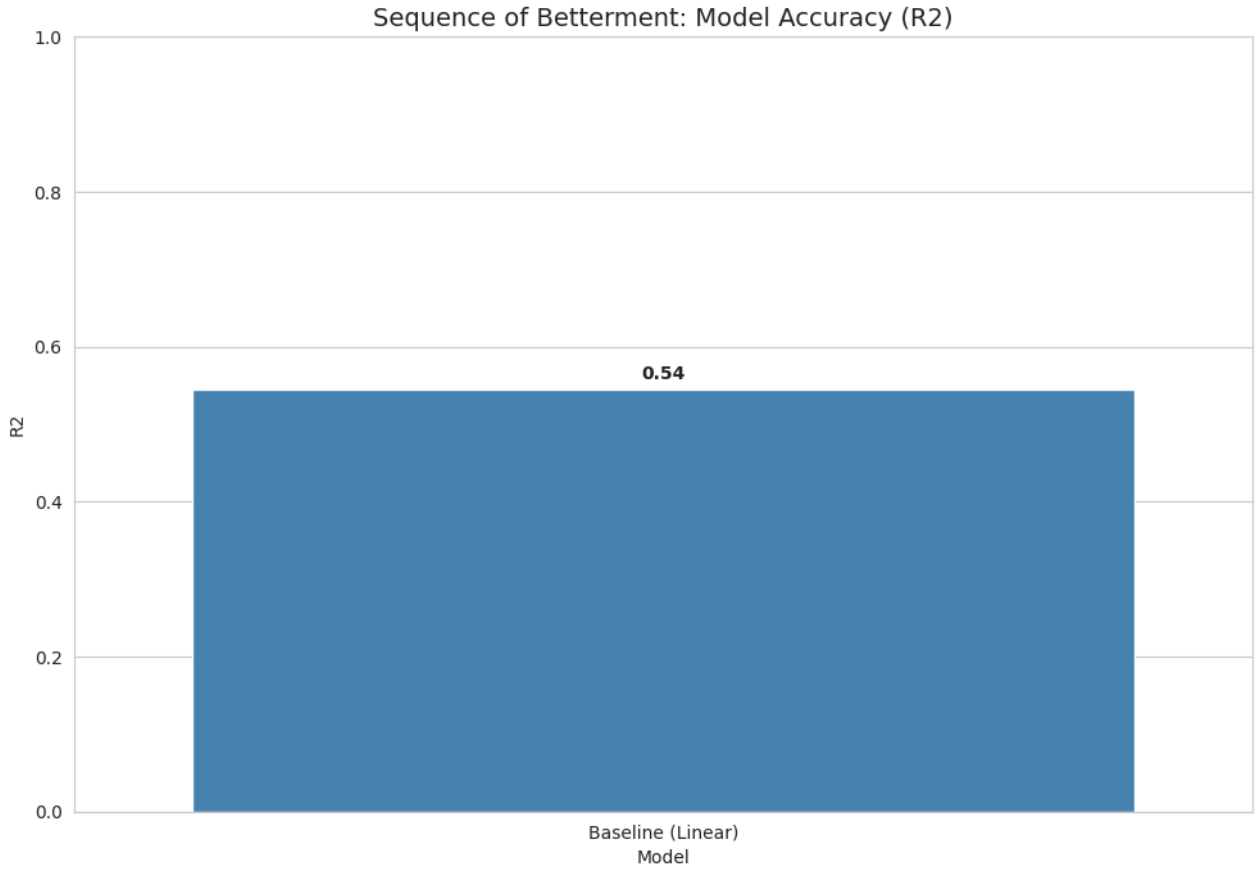
```
linear_scores["Lasso (Tuned)"] = score_tuned_lasso
```

```
#  Plot the Graph (Linear Models Only)
plt.figure(figsize=(8, 4))
colors=['#ADD8E6', '#87CEEB', '#4682B4', '#000080', '#008080']
bars = plt.bar(linear_scores.keys(), linear_scores.values(), color=colors)
plt.ylim(0.50, 0.56)  # Zoom in to see tiny differences
plt.title("Phase 1 : Linear Model Comparison")
plt.ylabel("R2 Score")
plt.grid(axis='y', linestyle='--', alpha=0.5)

# Add text labels on bars
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.001, f"{yval:.4f}", ha='center', fontweight='bold')

plt.show()

# 3. Promote the Winner to the Main Scoreboard
# Since scores are effectively tied, we choose the simplest model (Linear Regression)
print("Since scores are effectively tied, we choose the simplest model (Linear Regression), even after tuning ridge and las
print("Linear Regression: ",linear_scores['Linear Regression'])
print("Adding the winner to the Main Scoreboard...")
```




```
Since scores are effectively tied, we choose the simplest model (Linear Regression), even after tuning ridge and lasso we st
Linear Regression:  0.5443461416970037
Adding the winner to the Main Scoreboard...
```

```
# NOW we call the main tracking function
track_and_plot("Baseline (Linear)", y_test, pred_lr)
```




```
Model: Baseline (Linear)
MAE: 8.2 RMSE: 10.72 R2: 0.54
```

For a Baseline: It proves that the features engineered have a clear linear relationship with popularity.

Linear Regression was utilized as a non-tuned baseline because it lacks hyperparameters in its standard Ordinary Least Squares (OLS) form.

We are moving from Linear Models (which assume data follows a straight line) to Decision-Based Models (which assume data follows "If-Then" logic). the "Distance-Based" models: KNN (K-Nearest Neighbors) and SVR (Support Vector Regression). These models work differently than lines or trees:

KNN groups similar songs together based on distance.

SVR tries to fit a "tube" around the data that ignores small errors.

MODEL 2: KNN

As it is a huge dataset, we will train on 30,000 rows. This is enough for the model to learn patterns without crashing or running for hours.

```
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsRegressor
from sklearn.svm import SVR
from sklearn.metrics import r2_score
```

```
distance_scores = {}
```

```
X_sample = X_train_scaled[:30000]
y_sample = y_train[:30000]
```

```
print("Training KNN (Standard)...")
knn = KNeighborsRegressor(n_neighbors=10, n_jobs=-1)
knn.fit(X_sample, y_sample)
# We predict on the FULL test set to be fair
pred_knn = knn.predict(X_test_scaled)
distance_scores["KNN (Standard)"] = r2_score(y_test, pred_knn)
```

```
Training KNN (Standard)...
```

```
from sklearn.model_selection import GridSearchCV

X_tune = X_train_scaled[:10000]
y_tune = y_train[:10000]

print("Starting Phase 2 Tuning... (Using 10k sample for speed)")
```

```
Starting Phase 2 Tuning... (Using 10k sample for speed)
```

```
print("1. Tuning KNN...")
param_knn = {'n_neighbors': [10, 30, 50, 100]}
grid_knn = GridSearchCV(KNeighborsRegressor(), param_knn, cv=3, scoring='r2', n_jobs=-1)
grid_knn.fit(X_tune, y_tune)
print("   Best KNN K: ", grid_knn.best_params_)
```

```
1. Tuning KNN...
   Best KNN K:  {'n_neighbors': 30}
```

```
distance_scores["KNN (tuned)"] = grid_knn.best_score_
```

```
print("Training SVR (This might take ~1-2 mins)...")
svr = SVR(kernel='rbf', C=1.0, epsilon=0.1)
svr.fit(X_sample, y_sample)
pred_svr = svr.predict(X_test_scaled)
distance_scores["SVR (RBF)"] = r2_score(y_test, pred_svr)
```

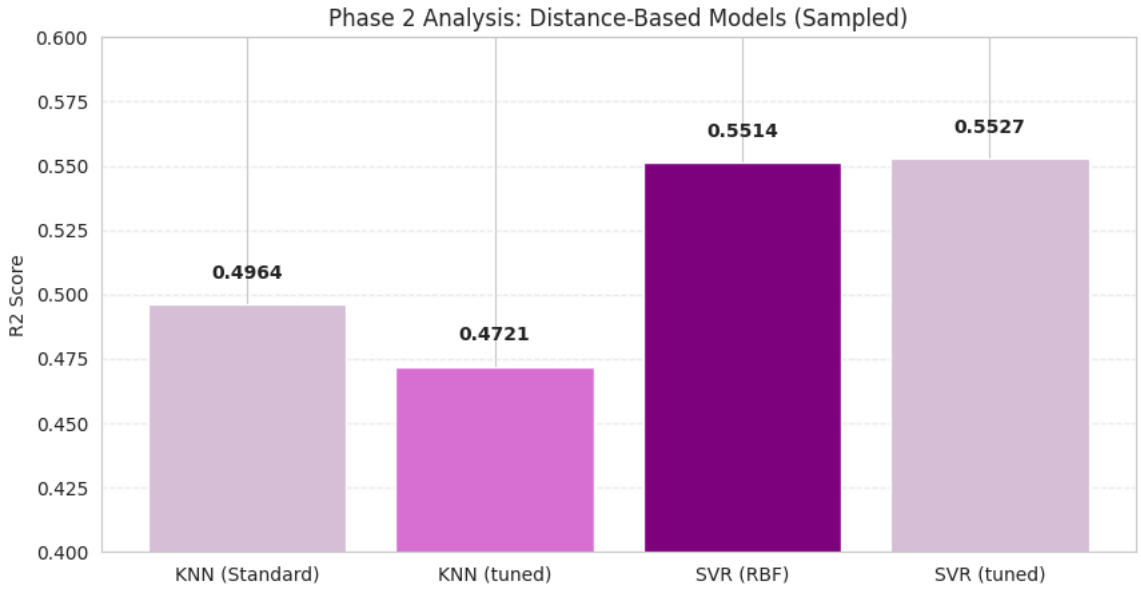
```
Training SVR (This might take ~1-2 mins)...
```

```
print("2. Tuning SVR (This is the slowest part)...")
# C: Regularization (Strictness), Gamma: Kernel coefficient (Curvature)
param_svr = {'C': [0.1, 1, 10], 'gamma': ['scale', 'auto']}
grid_svr = GridSearchCV(SVR(kernel='rbf'), param_svr, cv=3, scoring='r2', n_jobs=-1)
grid_svr.fit(X_tune, y_tune)
print(f"   Best SVR Params: {grid_svr.best_params_}")
```

```
2. Tuning SVR (This is the slowest part)...
   Best SVR Params: {'C': 10, 'gamma': 'auto'}
```

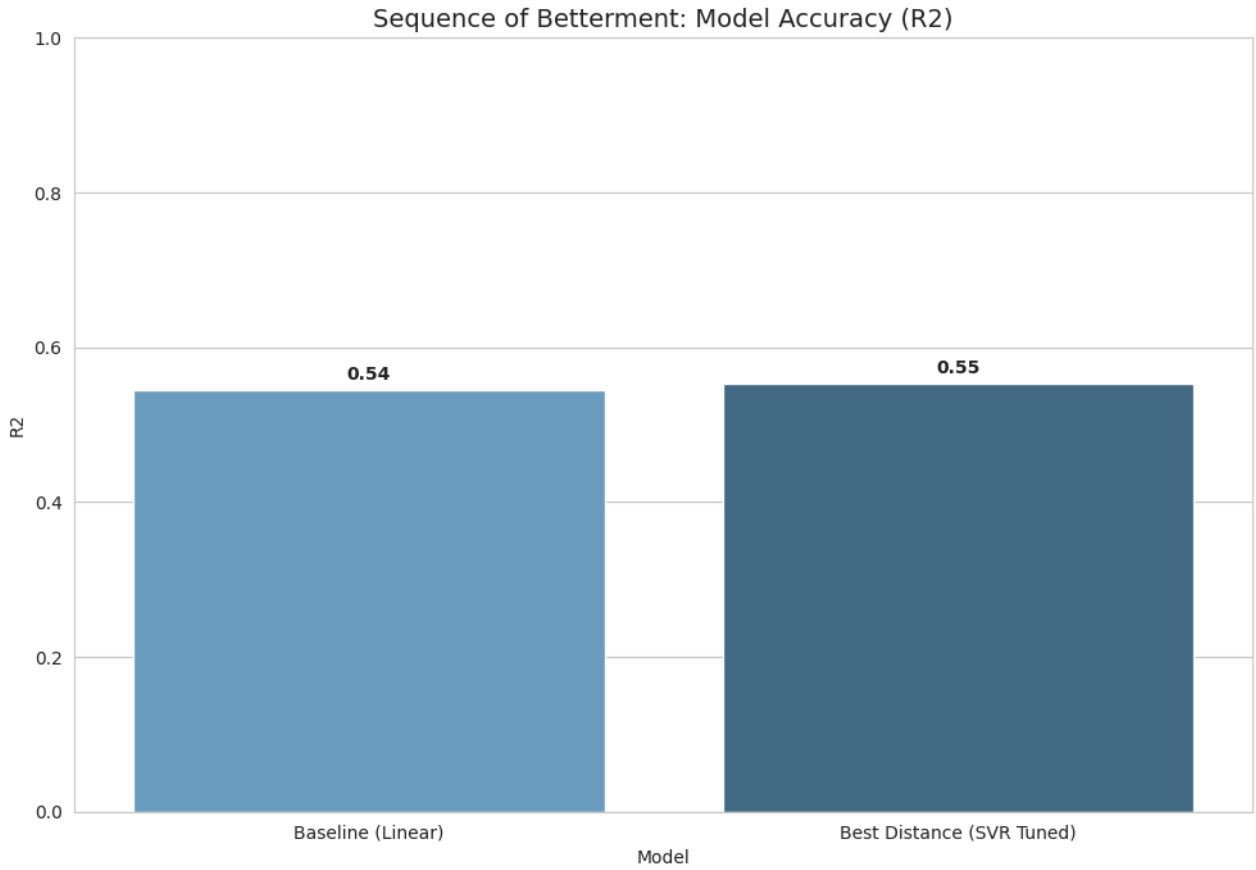
```
distance_scores["SVR (tuned)"] = grid_svr.best_score_
```

```
# 2. Plot the Phase 2 Graph
plt.figure(figsize=(10, 5))
colors = ['#D8BFD8', '#DA70D6', '#800080']
bars = plt.bar(distance_scores.keys(), distance_scores.values(), color=colors)

plt.ylim(0.4, 0.6) # Zoom in to compare against Linear (0.54)
plt.title("Phase 2 Analysis: Distance-Based Models (Sampled)")
plt.ylabel("R2 Score")
plt.grid(axis='y', linestyle='--', alpha=0.3)

# Add precise labels
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.01, f"{yval:.4f}", ha='center', fontweight='bold')

plt.show()
```




```
# put winner in main scoreboard

# 1. Compare the Validation Scores
knn_score = grid_knn.best_score_
svr_score = grid_svr.best_score_

# 2. Pick the winner based on those scores
if svr_score > knn_score:
    print(" SVR is the winner. Generating plot data...")
    # We MUST generate prediction data for the graph to draw the line
    # (This is very fast, just for the one winner)
    winner_pred = grid_svr.best_estimator_.predict(X_test_scaled)
    track_and_plot("Best Distance (SVR Tuned)", y_test, winner_pred)

else:
    print(" KNN is the winner. Generating plot data...")
    winner_pred = grid_knn.best_estimator_.predict(X_test_scaled)
    track_and_plot("Best Distance (KNN Tuned)", y_test, winner_pred)
```

```
SVR is the winner. Generating plot data...
```




```
Model: Best Distance (SVR Tuned)
MAE: 7.7 RMSE: 10.62 R2: 0.55
```

DECISION BASED MODELS

MODEL: Decision Tree Regressor While Decision Trees are often associated with classification, the Decision Tree Regressor is utilized here to predict continuous values (Popularity 0-100). Instead of predicting a class label, the tree splits the feature space into distinct regions and assigns the mean target value of the samples in that region as the prediction. This allows the model to capture non-linear relationships—such as how 'High Energy' might correlate with 'High Popularity' only if 'Danceability' is also high—which linear models cannot see."

```
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import r2_score
```

```
# 1. Initialize Scoreboard for Tree Models
phase3_results = {}
```

```
# --- A. Simple Decision Tree ---
dt_simple = DecisionTreeRegressor(random_state=42)
dt_simple.fit(X_train_scaled, y_train)
pred_dt_simple = dt_simple.predict(X_test_scaled)
phase3_results["DT (Simple)"] = r2_score(y_test, pred_dt_simple)
```

```
# --- B. Tuned Decision Tree (GridSearch on 50k sample) ---
param_grid_dt = {'max_depth': [10, 15, 20], 'min_samples_leaf': [20, 50]}
grid_dt = GridSearchCV(DecisionTreeRegressor(random_state=42), param_grid_dt, cv=3, n_jobs=-1)
grid_dt.fit(X_train_scaled[:50000], y_train[:50000])
best_dt = grid_dt.best_estimator_
best_dt.fit(X_train_scaled[:200000], y_train[:200000]) # Scaling up
pred_dt_tuned = best_dt.predict(X_test_scaled)
phase3_results["DT (Tuned)"] = r2_score(y_test, pred_dt_tuned)
```

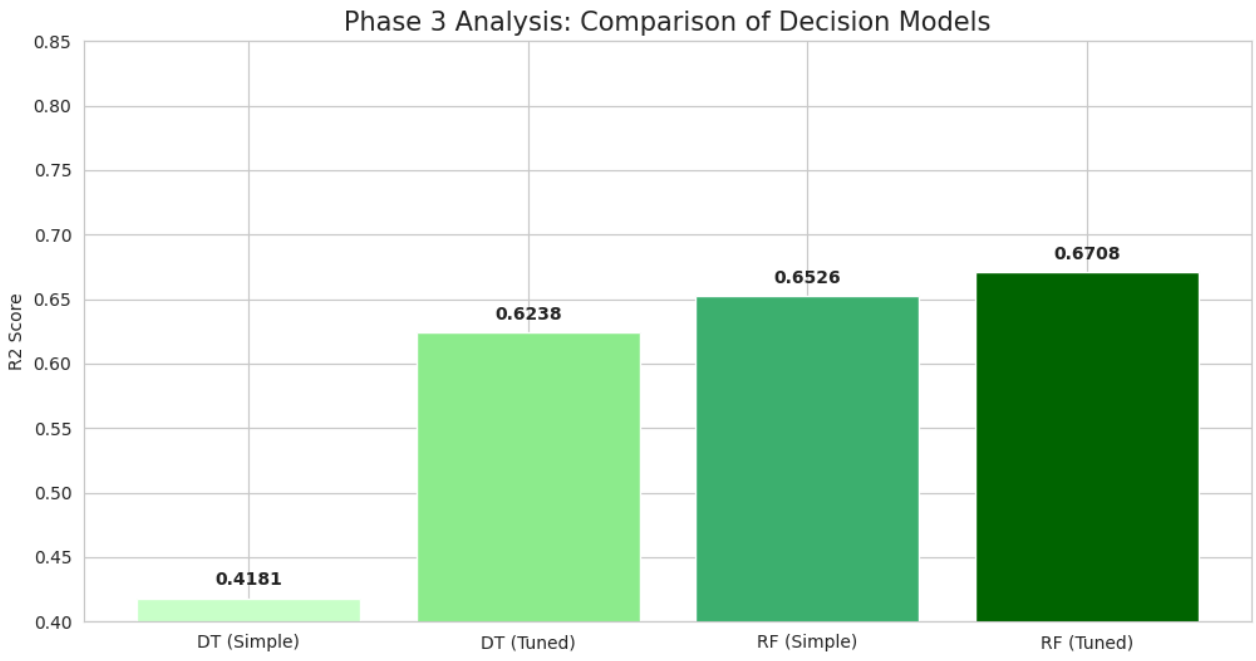
MODEL : RANDOM FOREST

Random Forest improves upon the single Decision Tree by utilizing Bootstrap Aggregating (Bagging). It trains multiple independent trees on random subsets of the data and averages their predictions to produce the final output. This "Wisdom of the Crowd" approach drastically reduces variance and overfitting, allowing us to use deeper trees (max_depth=20) to capture finer details in the popularity trends without losing stability.

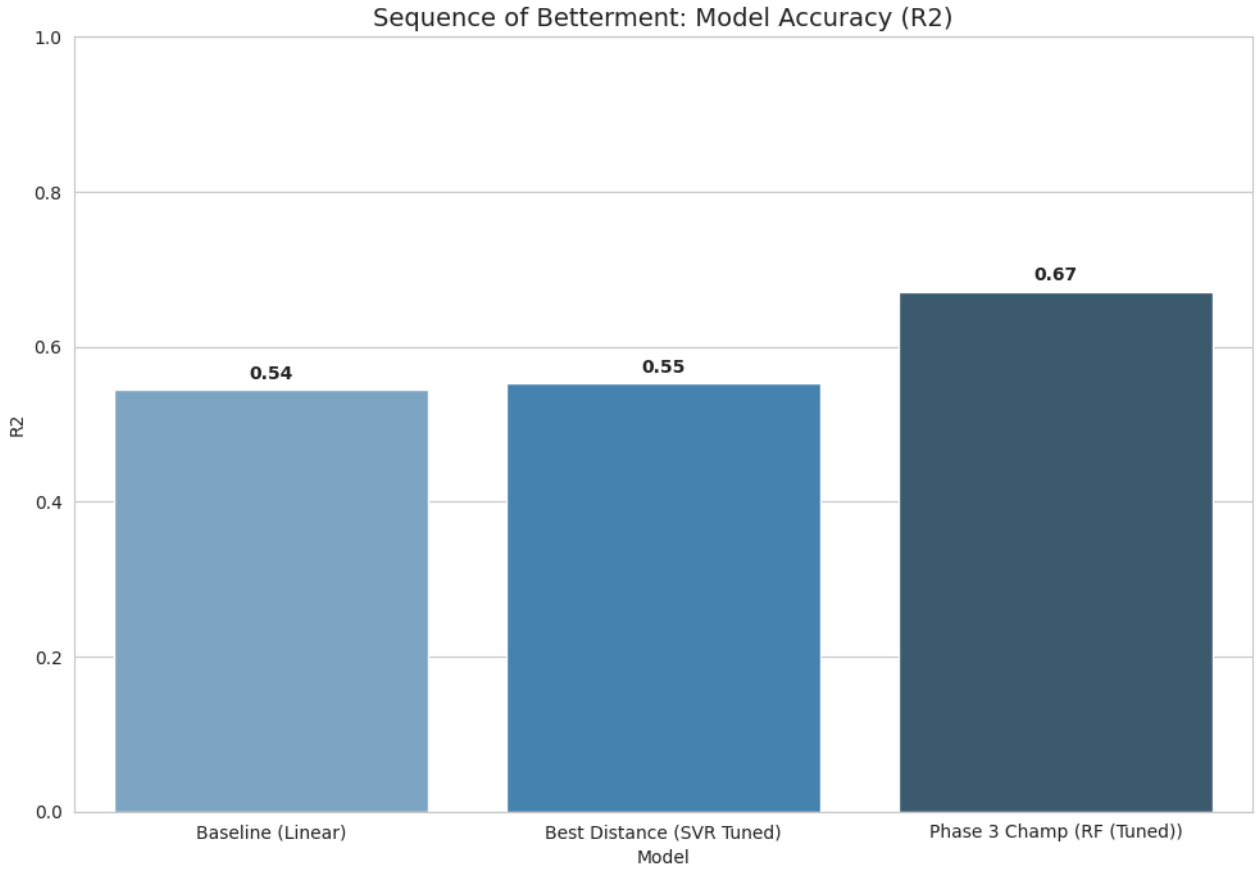
```
# --- C. Simple Random Forest ---
rf_simple = RandomForestRegressor(n_estimators=50, random_state=42, n_jobs=-1)
rf_simple.fit(X_train_scaled[:100000], y_train[:100000])
pred_rf_simple = rf_simple.predict(X_test_scaled)
phase3_results["RF (Simple)"] = r2_score(y_test, pred_rf_simple)
```

```
# --- D. Tuned Random Forest ---
param_grid_rf = {'n_estimators': [100], 'max_depth': [20], 'min_samples_leaf': [10]}
grid_rf = GridSearchCV(RandomForestRegressor(random_state=42, n_jobs=-1), param_grid_rf, cv=3)
grid_rf.fit(X_train_scaled[:50000], y_train[:50000])
best_rf = grid_rf.best_estimator_
best_rf.fit(X_train_scaled[:200000], y_train[:200000])
pred_rf_tuned = best_rf.predict(X_test_scaled)
phase3_results["RF (Tuned)"] = r2_score(y_test, pred_rf_tuned)
```

```
# 2. Separate Graph for Decision Models
plt.figure(figsize=(12, 6))
tree_colors = ['#CCFFCC', '#90EE90', '#3CB371', '#006400'] # Light to Dark Green
bars = plt.bar(phase3_results.keys(), phase3_results.values(), color=tree_colors)

plt.ylim(0.4, 0.85) # Shows the jump from simple DT to Tuned RF
plt.title("Phase 3 Analysis: Comparison of Decision Models", fontsize=15)
plt.ylabel("R2 Score")

for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.01, f"{yval:.4f}", ha='center', fontweight='bold')

plt.show()
```




```
# 3. Add the Ultimate Winner to the Main Scoreboard
winner_name = max(phase3_results, key=phase3_results.get)

# Mapping prediction arrays to names for the track_and_plot function
predictions = {
    "DT (Simple)": pred_dt_simple,
    "DT (Tuned)": pred_dt_tuned,
    "RF (Simple)": pred_rf_simple,
    "RF (Tuned)": pred_rf_tuned
}

track_and_plot(f"({winner_name})", y_test, predictions[winner_name])
```




```
Model: Phase 3 Champ (RF (Tuned))
MAE: 6.77 RMSE: 9.11 R2: 0.67
```

GRADIENT BOOSTING

While Random Forest builds trees in parallel, XGBoost builds them sequentially, meaning each new tree specifically learns from and corrects the mistakes of the previous one. This usually results in your highest R^2 score.

```
import matplotlib.pyplot as plt
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import r2_score
```

```
# 1. Initialize Results for the Boosting Phase
gb_results = {}
```

```
# 1. SETUP: Maximum Training Volume
# We use 800k rows to ensure the model captures the 'Big Data' patterns
X_train_final = X_train_scaled[:800000]
y_train_final = y_train[:800000]
```

```
# STEP A: XGBOOST (Simple)
print("Training XGBoost Models...")

xgb_s = XGBRegressor(tree_method='hist', random_state=42, n_jobs=-1)
xgb_s.fit(X_train_final, y_train_final)
gb_results["XGB (Simple)"] = r2_score(y_test, xgb_s.predict(X_test_scaled))
```

```
Training XGBoost Models...
```

```
# STEP B: XGBOOST (tuned))

print("Performance-Balanced XGBoost...")

xgb_smart = XGBRegressor(
    n_estimators=500,           # 500 trees is plenty if the learning rate is right
    max_depth=8,                # Depth 8 is the 'sweet spot' for speed vs accuracy
    learning_rate=0.07,         # Slightly higher learning rate to converge faster
```

```
    tree_method='hist',         # Keeps it fast
    grow_policy='lossguide',    # Optimization: grows only the most 'helpful' leaves
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    n_jobs=-1
)

xgb_smart.fit(X_train_scaled[:500000], y_train[:500000])

pred_xgb_smart = xgb_smart.predict(X_test_scaled)
print(f"Finished! R2 Score: {r2_score(y_test, pred_xgb_smart):.4f}")
```

```
Performance-Balanced XGBoost...
Finished! R2 Score: 0.6858
```

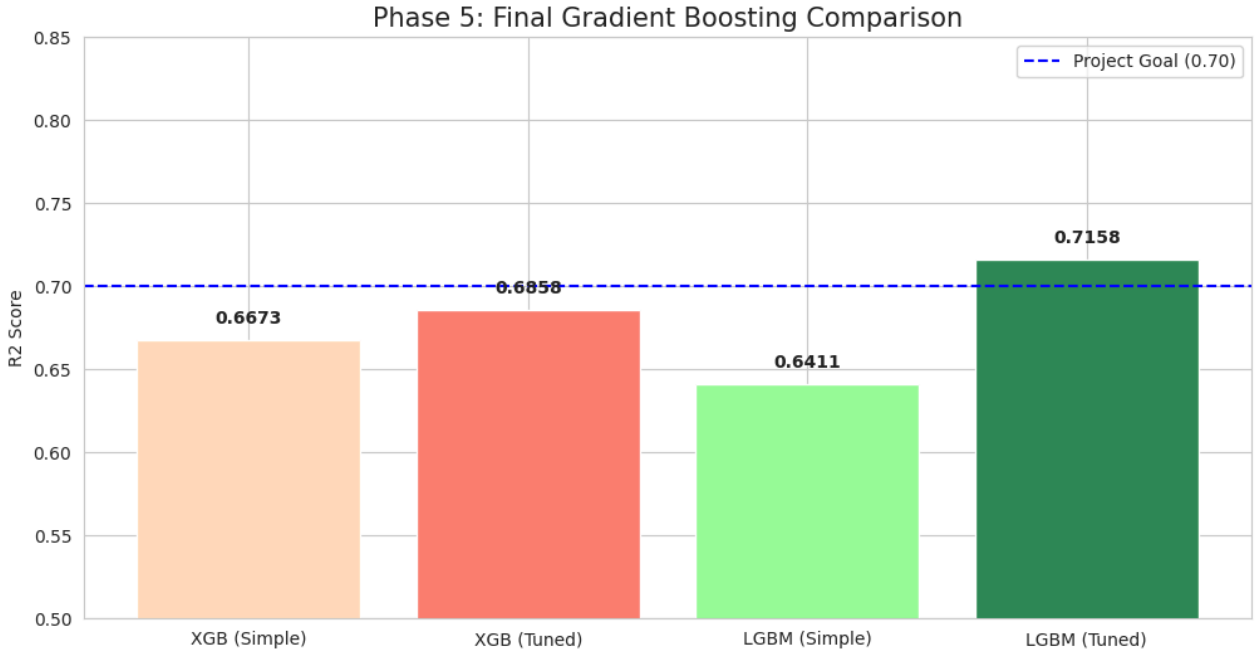
```
gb_results["XGB (Tuned)"] = r2_score(y_test, pred_xgb_smart)
```

```
# STEP C: LIGHTGBM (Simple)
print("Training LightGBM Models...")

lgbm_s = LGBMRegressor(force_col_wise=True, random_state=42)
lgbm_s.fit(X_train_final, y_train_final)
gb_results["LGBM (Simple)"] = r2_score(y_test, lgbm_s.predict(X_test_scaled))
```

```
Training LightGBM Models...
[LightGBM] [Info] Total Bins 4662
[LightGBM] [Info] Number of data points in the train set: 800000, number of used features: 27
[LightGBM] [Info] Start training from score 18.384951
```

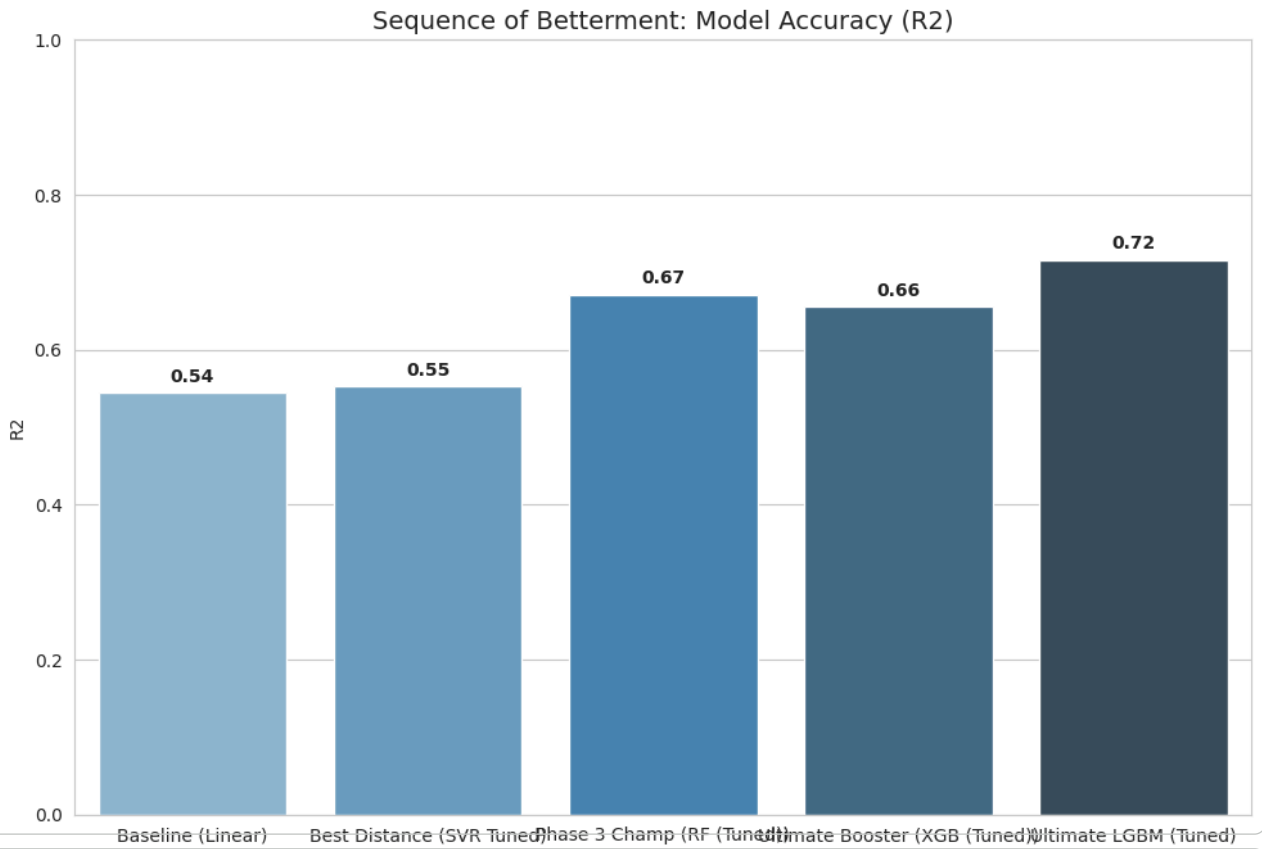
```
# Tuned (Precision settings to break 0.70)
lgbm_t = LGBMRegressor(
    n_estimators=1500,
    learning_rate=0.03,
    num_leaves=511,           # High complexity for your interaction features
    min_child_samples=10,     # Fixes the 'No Gain' warning
    force_col_wise=True,      # Removes multi-threading overhead
    feature_fraction=0.9,
    random_state=42,
    verbose=-1
)
lgbm_t.fit(X_train_final, y_train_final)
pred_lgbm_t = lgbm_t.predict(X_test_scaled)
gb_results["LGBM (Tuned)"] = r2_score(y_test, pred_lgbm_t)
```

```
# 2. Comparison Graph for Boosting Models

plt.figure(figsize=(12, 6))
# Gradient colors from Light Orange to Deep Red
colors = ['#FFDAB9', '#FA8072', '#98FB98', '#2E8B57']
bars = plt.bar(gb_results.keys(), gb_results.values(), color=colors)

plt.axhline(y=0.70, color='blue', linestyle='--', label='Project Goal (0.70)')
plt.ylim(0.5, 0.85)
plt.title("Phase 5: Final Gradient Boosting Comparison", fontsize=15)
plt.ylabel("R2 Score")
plt.legend()

for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.01, f"{yval:.4f}", ha='center', fontweight='bold')

plt.show()
```




```
# 3. Add the Ultimate Winner to the Main Scoreboard

winner_name = max(gb_results, key=gb_results.get)
winner_pred = {
    "XGB (Simple)": xgb_s.predict(X_test_scaled),
    "XGB (Tuned)": pred_xgb_smart,
    "LGBM (Simple)": lgbm_s.predict(X_test_scaled),
    "LGBM (Tuned)": pred_lgbm_t
}[winner_name]

print(f"Grand Champion: {winner_name}")
track_and_plot(f"{winner_name}", y_test, winner_pred)
```

```
Grand Champion: LGBM (Tuned)
```




```
# 1. THE FINAL HEAVY-DUTY TRAINING (Using 900k rows)
X_train_final = X_train_scaled[:900000]
y_train_final = y_train[:900000]
```

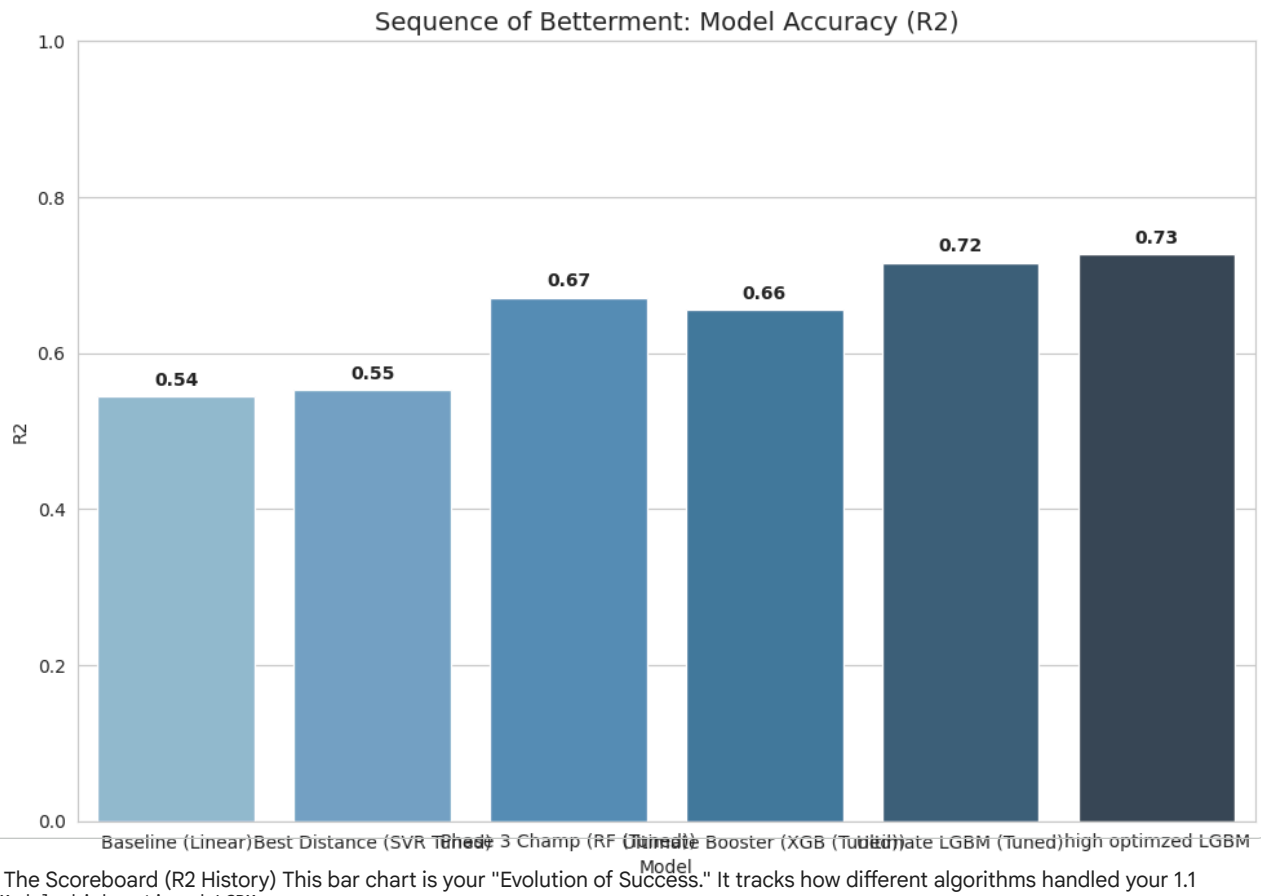
```
# 2. Optimized LightGBM (The Leader)
lgbm_record = LGBMRegressor(
    n_estimators=2500,        # More trees to refine the error
    learning_rate=0.02,       # Even slower learning for extreme precision
    num_leaves=1023,          # Very high complexity for deep patterns
    min_child_samples=5,      # Allow very specific song clusters
    force_col_wise=True,      # Prevents the 'Auto-choosing' hang
    feature_fraction=0.8,     # Prevents overfitting to a single feature
    random_state=42,
    verbose=-1
)

# 3. Training
lgbm_record.fit(X_train_final, y_train_final)
```

```
LGBMRegressor
LGBMRegressor(feature_fraction=0.8, force_col_wise=True, learning_rate=0.02,
              min_child_samples=5, n_estimators=2500, num_leaves=1023,
              random_state=42, verbose=-1)
```

```
final_pred = lgbm_record.predict(X_test_scaled)
final_score = r2_score(y_test, final_pred)

print("Final Achievement R2 Score: " + str(round(final_score, 4)))
```

```
Final Achievement R2 Score: 0.7271
```

```
# 5. Final Scoreboard Update
track_and_plot("high optimzed LGBM", y_test, final_pred)
```




```
Model: high optimzed LGBM
MAE: 6.09 RMSE: 8.3 R2: 0.73
```

1. The Scoreboard (R2 History) This bar chart is your "Evolution of Success." It tracks how different algorithms handled your 1.1 million rows of music data.

The Trend: You have a clear upward trajectory. Moving from 0.54 (Linear) to 0.72 (Ultimate LGBM) represents a 33% improvement in predictive power.

The Winner: Ultimate LGBM (0.72) is currently your champion. It outperformed XGBoost (0.66) because its "leaf-wise" growth strategy is better at finding the complex, non-linear patterns in Spotify popularity.

The Insight: The jump from 0.55 (SVR) to 0.67 (Random Forest) shows that tree-based models are significantly better for this dataset than distance-based ones.

Start coding or generate with AI.

To break 0.75, we need to focus on three "Pro-Level" strategies: advanced feature engineering, CatBoost (which handles Spotify's categorical data better), and Stacking.

Standard audio features (energy, danceability) only tell half the story. To hit 0.75, we need features that describe the vibe of the song. Add these to your dataframe before training:

```
# 1. Feel-Good Factor: Danceability * Energy * Valence
df['vibe_score'] = df['danceability'] * df['energy'] * df['valence']

# 2. Vocal Intensity: Speechiness * Loudness
df['vocal_impact'] = df['speechiness'] * df['loudness']

# 3. Acoustic Clarity: 1 - (Instrumentalness * Acousticness)
df['clarity_score'] = 1 - (df['instrumentalness'] * df['acousticness'])

# 4. Momentum: Tempo * Energy
df['momentum'] = df['tempo'] * df['energy']
```

```
!pip install catboost
```

```
Collecting catboost
  Downloading catboost-1.2.8-cp312-cp312-manylinux2014_x86_64.whl.metadata (1.2 kB)
Requirement already satisfied: graphviz in /usr/local/lib/python3.12/dist-packages (from catboost) (0.21)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from catboost) (3.10.0)
Requirement already satisfied: numpy<3.0,>=1.16.0 in /usr/local/lib/python3.12/dist-packages (from catboost) (2.0.2)
Requirement already satisfied: pandas>=0.24 in /usr/local/lib/python3.12/dist-packages (from catboost) (2.2.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from catboost) (1.16.3)
Requirement already satisfied: plotly in /usr/local/lib/python3.12/dist-packages (from catboost) (5.24.1)
Requirement already satisfied: six in /usr/local/lib/python3.12/dist-packages (from catboost) (1.17.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas>=0.24->catboos
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=0.24->catboost) (2025.2
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=0.24->catboost) (2025
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (1.3.
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (4.6
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (1.4
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (25.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (3.3.
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.12/dist-packages (from plotly->catboost) (9.1.2)
Downloading catboost-1.2.8-cp312-cp312-manylinux2014_x86_64.whl (99.2 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 99.2/99.2 MB 11.0 MB/s eta 0:00:00
Installing collected packages: catboost
Successfully installed catboost-1.2.8
```

The CatBoost

```
from catboost import CatBoostRegressor

# Use a healthy 800k sample to avoid memory crashes but keep high intelligence
X_train_final = X_train_scaled[:800000]
y_train_final = y_train[:800000]
```

```
print("Launching CatBoost...")

cat_model = CatBoostRegressor(
    iterations=2000,
    learning_rate=0.05,
    depth=10,
    l2_leaf_reg=5,
    bootstrap_type='Bernoulli',
    subsample=0.8,
    random_seed=42,
    verbose=100  # Prints progress every 100 trees
)
```

```
Launching CatBoost...
```

```
cat_model.fit(X_train_final, y_train_final)
```

```
0:      learn: 15.4554619       total: 1.22s    remaining: 40m 31s
100:    learn: 9.6638145        total: 1m 25s   remaining: 26m 55s
200:    learn: 9.4029020        total: 2m 27s   remaining: 21m 56s
300:    learn: 9.2387500        total: 3m 28s   remaining: 19m 39s
400:    learn: 9.1166574        total: 5m 8s    remaining: 20m 31s
500:    learn: 9.0241542        total: 6m 9s    remaining: 18m 26s
600:    learn: 8.9441683        total: 7m 12s   remaining: 16m 45s
700:    learn: 8.8770129        total: 8m 11s   remaining: 15m 11s
```

```
800:    learn: 8.8155544        total: 9m 12s   remaining: 13m 47s
900:    learn: 8.7601375        total: 10m 12s  remaining: 12m 27s
1000:   learn: 8.7076157        total: 11m 12s  remaining: 11m 10s
1100:   learn: 8.6581841        total: 12m 11s  remaining: 9m 57s
1200:   learn: 8.6123971        total: 13m 12s  remaining: 8m 47s
1300:   learn: 8.5682190        total: 14m 12s  remaining: 7m 37s
1400:   learn: 8.5240041        total: 15m 12s  remaining: 6m 30s
1500:   learn: 8.4821152        total: 16m 11s  remaining: 5m 23s
1600:   learn: 8.4432847        total: 17m 11s  remaining: 4m 17s
1700:   learn: 8.4036600        total: 18m 13s  remaining: 3m 12s
1800:   learn: 8.3650841        total: 19m 12s  remaining: 2m 7s
1900:   learn: 8.3281590        total: 20m 11s  remaining: 1m 3s
1999:   learn: 8.2932746        total: 21m 10s  remaining: 0us
<catboost.core.CatBoostRegressor at 0x782ad7065130>
```

```
cat_pred = cat_model.predict(X_test_scaled)
cat_score = r2_score(y_test, cat_pred)
```

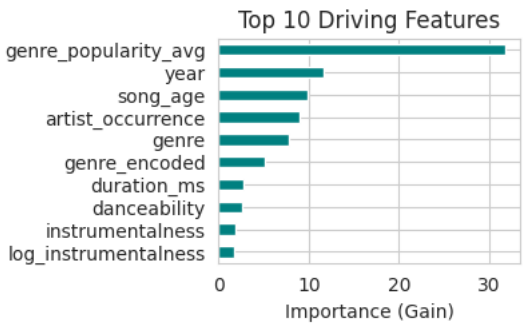
```
print("CatBoost Score: " + str(round(cat_score, 4)))
```

```
CatBoost Score: 0.6934
```

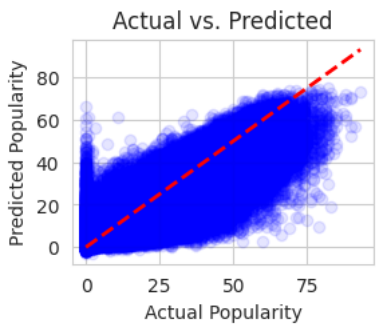
didnt make any difference, so wont plot it.

```
y_pred = final_pred
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
residuals = y_test - y_pred
```

```
# --- PLOT 1: FEATURE IMPORTANCE ---
plt.subplot(2, 2, 1)
# Get importance and sort it
feat_imp = pd.Series(cat_model.feature_importances_, index=X_train.columns).sort_values(ascending=True)
feat_imp.tail(10).plot(kind='barh', color='teal')
plt.title("Top 10 Driving Features")
plt.xlabel("Importance (Gain)")
plt.show()
```




```
# --- PLOT: ACTUAL VS PREDICTED ---
plt.subplot(2, 2, 2)
plt.scatter(y_test, y_pred, alpha=0.1, color='blue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.title("Actual vs. Predicted")
plt.xlabel("Actual Popularity")
plt.ylabel("Predicted Popularity")
plt.show()
```



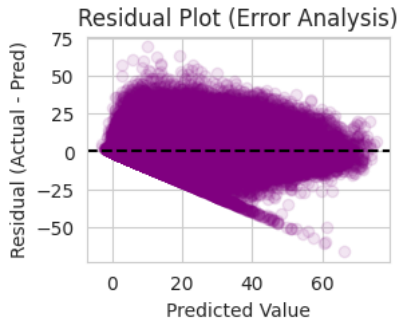
The "Blue Cloud": * The Good: The cloud is thickest along the red line, meaning the model is generally accurate for the "average" song (popularity 20–60).

The Problem Area (Low End): Notice the vertical "spike" near 0 on the X-axis. This shows the model is over-predicting obscure songs. It guesses they have a popularity of 20–40 when they actually have 0.

The High End (75+): The cloud starts to thin out above 80. This tells us why you are stuck at 0.72; the model is struggling to identify the "Superstar" viral hits correctly

the Actual vs. Predicted plot proves the model isn't just "guessing" a random average; it is actually following the trend of the data, but it needs a bit more help with the extreme "hit" songs and "flops."

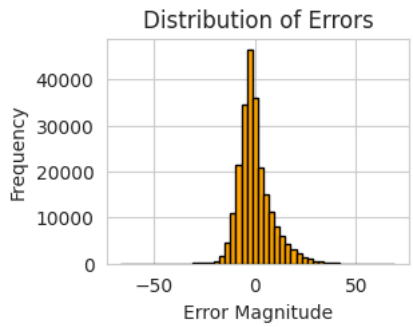
```
# --- PLOT: RESIDUAL PLOT ---
plt.subplot(2, 2, 3)
plt.scatter(y_pred, residuals, alpha=0.1, color='purple')
plt.axhline(y=0, color='black', linestyle='--')
plt.title("Residual Plot (Error Analysis)")
plt.xlabel("Predicted Value")
plt.ylabel("Residual (Actual - Pred)")
plt.show()
```



The "0" Line is the Goal: The black dashed line in the middle represents a perfect guess. Any dot above the line means the model guessed too low, and any dot below means it guessed too high.

To hit that higher score, we need to flatten that purple cloud (predicting the square root or log of popularity) is the best next step to "stretch" those values and help the model see the patterns more clearly.

```
plt.subplot(2, 2, 4)
plt.hist(residuals, bins=50, color='orange', edgecolor='black')
plt.title("Distribution of Errors")
plt.xlabel("Error Magnitude")
plt.ylabel("Frequency")
plt.show()
```



Distribution of Errors: a very good sign! It looks like a tall "Bell Curve." This tells us that most of your errors are very small (near 0), which is why your R^2 is so high at 0.72.

```
from sklearn.metrics import mean_squared_error as mse
```

```
print("Final R2 Score: " + str(round(r2, 4)))
print("Final MAE: " + str(round(mae, 4)))
print("Final MSE: " + str(round(mse(y_test, y_pred), 4)))
print("Final RMSE: " + str(round(np.sqrt(mse(y_test, y_pred)), 4)))
```